Fiche de révision du projet MosaicArt

Yan Rabefaniraka

2026-11-09

Table des matières

1 1. Objectif du projet . . . **2**
2 2. Conventions de coordonnées . . . **3**
3 3. Détection des coins de Harris . . . **3**
3.1 Fonction . . . 3
3.2 Gradients de l'image . . . 4
3.3 Termes du tenseur de structure . . . 4
3.4 Suppression des non-maxima locaux . . . 4
3.5 Sélection des points . . . 5
3.6 Cas particuliers importants . . . 5
4 4. Extraction des voisinages . . . **5**
4.1 Fonction . . . 5
4.2 Extraction . . . 6
4.3 Conditions importantes . . . 6
4.4 Erreur fréquente . . . 6
5 5. Corrélation ZNCC . . . **6**
5.1 Fonction . . . 6
5.2 Points valides . . . 6
5.3 Voisinages centrés . . . 7
5.4 Covariance . . . 7
5.5 Variance . . . 7
5.6 ZNCC . . . 7
5.7 Problème numérique important . . . 7
6 6. Appariement des points . . . **7**
6.1 Fonction . . . 8
6.2 Méthode du meilleur candidat . . . 8
6.3 Contrainte de symétrie . . . 8
6.4 Contrainte de seuillage . . . 8
6.5 Sortie . . . 8
6.6 Limites . . . 9
7 7. Estimation de l'homographie . . . **9**
7.1 Objectif . . . 9
7.2 Nombre minimal de points . . . 9
7.3 Système linéaire . . . 9
7.4 Résolution par SVD . . . 10
7.5 Configurations dégénérées . . . 10
7.6 Conditionnement numérique . . . 10
8 8. Application d'une homographie . . . **10**
8.1 Fonction . . . 10
8.2 Passage en coordonnées homogènes . . . 10
8.3 Transformation matricielle . . . 10
8.4 Normalisation projective . . . 10
8.5 Distinction importante . . . 11

9 9. Construction d'une mosaïque en niveaux de gris 11
9.1 Sens de la transformation 11
9.2 Dimensions de la mosaïque 11
9.3 Translation dans le repère de la mosaïque 12
9.4 Déformation inverse 12
9.5 Différence entre les deux versions 12
9.6 Artefacts possibles 12
10 10. Mosaïque couleur 12
10.1 Différence principale 13
10.2 Sortie 13
10.3 Fusion des couleurs 13
10.4 Hypothèses 13
11 11. Chaîne complète de traitement 13
11.1 Lecture des images 13
11.2 Détection des points 14
11.3 Appariement des images 1 et 2 14
11.4 Estimation de l'homographie 14
11.5 Construction des mosaïques à deux images 14
11.6 Construction d'une mosaïque à trois images 14
12 12. Concepts MATLAB essentiels 15
12.1 Dimensions des matrices 15
12.2 Opérateurs terme à terme 15
12.3 Opérateurs matriciels 15
12.4 Indexation linéaire 15
12.5 `ind2sub` 15
12.6 Conversion des images 15
13 13. Pièges fréquents à l'examen 16
14 Questionnaire détaillé 16
14.1 Questionnaire A — Détection de Harris 16
14.2 Questionnaire B — Extraction des voisinages 17
14.3 Questionnaire C — ZNCC et appariement des points 17
14.4 Questionnaire D — Estimation de l'homographie 18
14.5 Questionnaire E — Application d'une homographie 18
14.6 Questionnaire F — Mosaïques en niveaux de gris 19
14.7 Questionnaire G — Mosaïques couleur 19
14.8 Questionnaire H — Composition de trois images 20
15 Liste de vérification finale 20

1 1. Objectif du projet

Construire des mosaïques d'images à partir de photographies présentant une zone de recouvrement :

1. Détecter les points d'intérêt avec le détecteur de Harris.
2. Extraire les voisinages locaux.
3. Apparier les points à l'aide de la corrélation ZNCC.
4. Estimer une homographie.
5. Déformer et fusionner les images.
6. Étendre la méthode aux images couleur et aux mosaïques de trois images.

Chaîne principale :

```
Images
  → Points d'intérêt de Harris
  → Appariement des points par ZNCC
  → Estimation de l'homographie
```

```
  → Déformation des images
  → Construction de la mosaïque
```

---

2 2. Conventions de coordonnées

Indexation des images dans MATLAB :

```
I(y, x)
```

- `x` : colonne, coordonnée horizontale.
- `y` : ligne, coordonnée verticale.
- Format d'un point :

```
xy = [x y]
```

Point en coordonnées homogènes :

```
[x y 1]
```

Transformation d'un point par `H` :

```
p2_h = H * p1_h
```

Puis normalisation :

```
x2 = p2_h(1) / p2_h(3);
y2 = p2_h(2) / p2_h(3);
```

---

3 3. Détection des coins de Harris

Fichier : `harris.m`

3.1 Fonction

```
[XY, R] = harris(Im, TailleFenetre, NbPoints, k)
```

Paramètres d'entrée :

- `Im` : image d'entrée.
- `TailleFenetre` : taille impaire de la fenêtre du voisinage.
- `NbPoints` : nombre maximal de points.
- `k` : paramètre de sensibilité de Harris.

Paramètres de sortie :

- `XY` : points détectés, au format `[x y]`.
- `R` : image des réponses de Harris.

Paramètres typiques :

```
TailleFenetre = 15;
NbPoints = 100;
k = 0.05;
```

3.2 Gradients de l'image

```
[Ii, Ij] = gradient(Im);
```

- `Ii` : dérivée selon les lignes, approximativement la direction verticale.
- `Ij` : dérivée selon les colonnes, approximativement la direction horizontale.

L'image est d'abord convertie :

```
Im = double(Im);
```

Cette conversion évite les erreurs liées aux calculs sur des entiers.

3.3 Termes du tenseur de structure

Les produits locaux des gradients sont :

```
Ij.^2
Ii.^2
Ij .* Ii
```

Ils sont lissés avec un filtre gaussien :

```
sig = (TailleFenetre - 1) / 4;
L = fspecial("gaussian", TailleFenetre, sig);
```

Puis :

```
A = conv2(Ij.^2, L, 'same');
B = conv2(Ii.^2, L, 'same');
C = conv2(Ij .* Ii, L, 'same');
```

La réponse de Harris est :

```
R = A .* B - C.^2 - k .* (A + B).^2;
```

Mathématiquement :

```
R = det(M) - k trace(M)^2
```

avec :

```
M = [ A C
      C B ]
```

Interprétation :

- Région uniforme : gradients faibles, réponse faible.
- Contour : gradient fort dans une direction, réponse généralement faible.
- Coin : gradients forts dans deux directions, réponse positive élevée.

3.4 Suppression des non-maxima locaux

Le voisinage complet ne peut pas être calculé sur les bords de l'image. Les réponses des bords reçoivent donc la valeur `-Inf` :

```
n2 = floor(TailleFenetre / 2);
```

Pour chaque pixel valide :

```
voisinage = R(i-n2:i+n2, j-n2:j+n2);
```

Le point est conservé si :

```
R(i,j) == max(voisinage, [], "all")
```

L'image obtenue est `Res`.

3.5 Sélection des points

Les réponses sont triées après linéarisation :

```
[~, is] = sort(Res(:), "descend");
```

Les valeurs invalides situées sur les bords sont supprimées :

```
is = is(~isinf(Res(is)));
```

Les coordonnées sont ensuite récupérées :

```
[Y, X] = ind2sub(size(R), is);
XY = [X, Y];
```

Détail important concernant MATLAB :

```
ind2sub → ligne, colonne
```

Ainsi :

```
Y = ligne
X = colonne
```

mais le projet stocke les coordonnées sous la forme :

```
[x y] = [colonne ligne]
```

3.6 Cas particuliers importants

- `TailleFenetre` doit être impaire.
- Les points proches des bords ne permettent pas d'extraire une fenêtre complète.
- `NbPoints` est limité au nombre de maxima locaux valides.
- Des maxima de même valeur peuvent produire plusieurs points sélectionnés sur un plateau.
- La réponse de Harris dépend fortement de l'échelle de l'image, du lissage et de `k`.

---

4 4. Extraction des voisinages

Fichier : `voisinage.m`

4.1 Fonction

```
voisins = voisinage(I, xyPt, TailleFenetre)
```

Paramètres d'entrée :

- `I` : image.
- `xyPt` : points au format `[x y]`.
- `TailleFenetre` : largeur et hauteur du voisinage.

Paramètre de sortie :

- Une fenêtre aplatie par ligne.

Pour `npt` points :

```
size(voisins) = [npt, TailleFenetre^2]
```

4.2 Extraction

```
K = floor(TailleFenetre / 2);
```

Pour le point `(x,y)` :

```
fenetre = I(y-K:y+K, x-K:x+K);
```

Le voisinage est aplati :

```
voisins(p,:) = fenetre(:)';
```

4.3 Conditions importantes

La fonction suppose que chaque point est suffisamment éloigné des bords :

```
x-K ⩾ 1
x+K ⩽ largeur de l'image
y-K ⩾ 1
y+K ⩽ hauteur de l'image
```

C'est pourquoi `apparier_Points.m` élimine les points invalides avant d'appeler cette fonction.

4.4 Erreur fréquente

Ne pas confondre :

```
I(x,y)
```

avec :

```
I(y,x)
```

MATLAB utilise une indexation ligne-colonne :

```
I(y,x)
```

---

5 5. Corrélation ZNCC

Fichier : `apparier_Points.m`

5.1 Fonction

```
C = apparier_Points(I1, Ptint1, I2, Ptint2, K)
```

La matrice de sortie vérifie :

```
C(i,j) = corrélation entre le point i de l'image 1
         et le point j de l'image 2
```

5.2 Points valides

Un point est valide si sa fenêtre complète est contenue dans l'image :

```
Ptint(:,1)-K ⩾ 1
Ptint(:,1)+K ⩽ largeur
```

```
Ptint(:,2)-K ⩾ 1
Ptint(:,2)+K ⩽ hauteur
```

5.3 Voisinages centrés

Pour chaque voisinage :

```
mean_gris1 = mean(vois1,2);
mean_gris2 = mean(vois2,2);
```

Les données centrées sont :

```
vois1 - mean_gris1
vois2 - mean_gris2
```

5.4 Covariance

```
cor = (vois1 - mean_gris1) * (vois2 - mean_gris2)' / NbPix;
```

Cette expression calcule efficacement toutes les covariances par multiplication matricielle.

5.5 Variance

```
var_gris1 = var(vois1,1,2);
var_gris2 = var(vois2,1,2);
```

Le deuxième argument `1` indique une normalisation par `N`, et non par `N-1`.

5.6 ZNCC

```
cor = cor ./ sqrt(var_gris1 * var_gris2');
```

La ZNCC est théoriquement comprise entre `-1` et `1` :

- `1` : motifs identiques, à un changement d'échelle de luminance près.
- `0` : absence de corrélation linéaire.
- `-1` : contraste inversé.

Le projet utilise ensuite :

```
C = abs(C);
```

Ainsi, les corrélations positives et négatives fortes sont considérées comme de bons appariements.

5.7 Problème numérique important

Un voisinage d'intensité constante possède une variance nulle :

```
sqrt(var1 * var2) = 0
```

Cela peut produire `NaN` ou `Inf`. Une implémentation robuste doit traiter explicitement les fenêtres de variance nulle.

---

6 6. Appariement des points

Fichier : `apparier_POI.m`

6.1 Fonction

```
[XY_C1, XY_C2] = apparier_POI( ...
    Im1, XY_1, Im2, XY_2, TailleFenetre, seuil)
```

6.2 Méthode du meilleur candidat

Pour chaque point de l'image 1 :

```
[~, ind1_2] = max(C');
ind1_2 = ind1_2(:);
```

`ind1_2(i)` est le meilleur candidat dans l'image 2 pour le point `i`.

Pour chaque point de l'image 2 :

```
[~, ind2_1] = max(C);
ind2_1 = ind2_1(:);
```

`ind2_1(j)` est le meilleur candidat dans l'image 1 pour le point `j`.

Cette méthode est souvent appelée **Winner Takes All** ou **WTA**.

6.3 Contrainte de symétrie

Un appariement est accepté uniquement si :

```
point 1 → point 2
et
point 2 → point 1
```

Code correspondant :

```
bidirectionnelle = ind2_1(ind1_2(ind1)) == ind1;
```

Il s'agit d'un appariement par plus proche voisin mutuel.

6.4 Contrainte de seuillage

Le score doit respecter :

```
score ⩾ seuil
```

Valeur typique :

```
seuil = 0.9;
```

6.5 Sortie

Les matrices finales de coordonnées sont :

```
XY_C1 = [mesurex1(:) mesurey1(:)];
XY_C2 = [mesurex2(:) mesurey2(:)];
```

Les lignes correspondent deux à deux :

```
XY_C1(i,:) <→ XY_C2(i,:)
```

6.6 Limites

- Aucun rejet géométrique des valeurs aberrantes n'est effectué.
- La méthode WTA peut sélectionner des appariements ambigus.
- Un seuil élevé peut supprimer trop de points.
- Un seuil faible peut introduire de mauvais appariements dans l'estimation de l'homographie.
- La corrélation absolue accepte également une corrélation photométrique négative.

---

7 7. Estimation de l'homographie

Fichier : `homographie.m`

7.1 Objectif

Estimer une transformation projective :

```
p2 ~ H p1
```

avec :

```
H = [h11 h12 h13
     h21 h22 h23
     h31 h32 h33]
```

L'homographie possède neuf coefficients, mais seulement huit degrés de liberté, car multiplier `H` par un scalaire non nul ne change pas la transformation.

7.2 Nombre minimal de points

Chaque correspondance fournit deux équations.

```
8 équations indépendantes / 2 équations par point = 4 points
```

Il faut donc au moins quatre paires de points non dégénérées.

7.3 Système linéaire

Pour chaque paire de points :

```
(x1,y1) → (x2,y2)
```

le programme construit deux lignes de la matrice `A`.

Le système complet est :

```
A h = 0
```

où `h` contient les coefficients de l'homographie.

L'implémentation construit :

```
bloc_1 = [x1 y1 ones(N,1) zeros(N,3);
          zeros(N,3) x1 y1 ones(N,1)];

bloc_2 = [-x2 .* x1 -x2 .* y1 -x2;
          -y2 .* x1 -y2 .* y1 -y2];

A = [bloc_1 bloc_2];
```

7.4 Résolution par SVD

```
[~,~,V] = svd(A);
H = V(:,end);
```

La dernière colonne de `V` correspond au vecteur associé à la plus petite valeur singulière.

Puis le vecteur est remis sous forme matricielle :

```
H = reshape(H,3,3)';
```

La transposition est essentielle, car MATLAB utilise un stockage des matrices par colonnes.

7.5 Configurations dégénérées

Quatre points ne suffisent pas s'ils sont mal répartis. Des problèmes apparaissent lorsque :

- les points sont alignés ;
- les points sont presque alignés ;
- les correspondances sont incorrectes ;
- la géométrie des images ne peut pas être représentée par une homographie unique.

7.6 Conditionnement numérique

Des coordonnées en pixels non normalisées peuvent produire un système mal conditionné. Une méthode plus robuste consiste à normaliser les coordonnées :

1. Translater les points pour obtenir une moyenne nulle.
2. Mettre à l'échelle la distance moyenne afin d'obtenir `sqrt(2)`.
3. Estimer l'homographie normalisée.
4. Dénormaliser l'homographie.

---

8 8. Application d'une homographie

Fichier : `appliquerHomographie.m`

8.1 Fonction

```
xy2 = appliquerHomographie(H, xy1)
```

Paramètres d'entrée :

- `H` : matrice `3 x 3`.
- `xy1` : points sous la forme `N x 2`.

8.2 Passage en coordonnées homogènes

```
xy1_homogene = [xy1 ones(NbPoints,1)];
```

8.3 Transformation matricielle

Les points sont transposés afin de permettre la multiplication matricielle :

```
xy2_3d = (H * xy1_homogene')';
```

8.4 Normalisation projective

```
z = xy2_3d(:,3);
xy2_homogene = xy2_3d ./ z;
xy2 = xy2_homogene(:,1:2);
```

Chaque ligne est divisée par sa propre troisième coordonnée.

8.5 Distinction importante

La fonction transforme des points de manière continue. Elle ne :

- fait pas d'interpolation des pixels ;
- ne vérifie pas les limites de l'image ;
- ne traite pas le cas `z = 0` ;
- n'arrondit pas les coordonnées.

L'arrondi est réalisé plus tard lors de la construction de la mosaïque.

---

9 9. Construction d'une mosaïque en niveaux de gris

Fichiers :

- `mosaique.m`
- `mosaiquebis.m`

9.1 Sens de la transformation

La matrice estimée transforme normalement :

```
image 1 → image 2
```

Pour construire une mosaïque, l'algorithme parcourt le repère de la mosaïque, ou de l'image 1, puis remonte vers l'image 2.

Il utilise donc :

```
Hinv = inv(H);
```

et applique :

```
xy_R2 = appliquerHomographie(H, xy_R1);
```

selon la convention adoptée.

Il faut toujours vérifier le sens de la transformation à partir d'un point connu.

9.2 Dimensions de la mosaïque

Les quatre coins de l'image 2 sont :

```
xy_coinsI2_R2 = [1 1;
                 nbcI2 1;
                 nbcI2 nblI2;
                 1 nblI2];
```

Ils sont projetés dans le repère de l'image 1.

Les limites de la mosaïque combinent :

- les limites de l'image 1 ;
- les limites de l'image 2 projetées.

```
xmin = floor(min( ... ));
ymin = floor(min( ... ));
xmax = ceil(max( ... ));
ymax = ceil(max( ... ));
```

Les dimensions résultantes sont :

```
nblImos = ymax-ymin+1;
nbcImos = xmax-xmin+1;
```

9.3 Translation dans le repère de la mosaïque

Si la coordonnée minimale n'est pas égale à 1, l'image 1 doit être décalée :

```
O1x_Rmos = 1 - (xmin - 1);
O1y_Rmos = 1 - (ymin - 1);
```

L'image est copiée dans :

```
Imos(O1y_Rmos: ... , O1x_Rmos: ... ) = I1;
```

9.4 Déformation inverse

Pour chaque pixel de la mosaïque :

1. Convertir les coordonnées de la mosaïque dans le repère de l'image 1.
2. Transformer ces coordonnées dans le repère de l'image 2.
3. Arrondir au pixel le plus proche.
4. Vérifier que le pixel appartient à l'image 2.
5. Copier sa valeur.

```
x_R2 = round(xy_R2(1));
y_R2 = round(xy_R2(2));
```

Il s'agit d'un échantillonnage par plus proche voisin.

9.5 Différence entre les deux versions

`mosaique.m` copie directement les pixels de l'image 2 dans le résultat.

`mosaiquebis.m` ajoute une fusion dans la zone de recouvrement :

```
d1 = nbcI1 - x_R1;
d2 = x_R2;

p1 = d1 / (d1+d2);
p2 = d2 / (d1+d2);

Imos(y,x) = p1*I1(y_R1,x_R1) + p2*I2(y_R2,x_R2);
```

La seconde version réduit les raccords visibles, mais ses poids dépendent uniquement de la distance horizontale.

9.6 Artefacts possibles

Les artefacts possibles comprennent :

- des trous noirs lorsqu'aucun pixel source n'est affecté ;
- des fissures causées par l'arrondi au plus proche voisin ;
- des raccords dus à une fusion incorrecte ;
- des images fantômes dues à de mauvais appariements ;
- des déformations dues à une homographie incorrecte ;
- des problèmes de division proches de l'infini projectif.

---

10 10. Mosaïque couleur

Fichier : `mosaiquecoul.m`

10.1 Différence principale

Une image couleur possède trois dimensions :

```
[nbl, nbc, nbCan] = size(I)
```

Généralement :

```
hauteur x largeur x 3
```

La fonction convertit les images en nombres flottants :

```
I1 = double(I1);
I2 = double(I2);
```

Cela évite les débordements sur les entiers et permet la fusion pondérée.

10.2 Sortie

```
Imos = zeros(nblImos, nbcImos, nbCan);
```

Les pixels sont affectés pour tous les canaux :

```
Imos(y,x,:) = ...
```

La même transformation géométrique est appliquée à chaque canal.

10.3 Fusion des couleurs

```
Imos(y,x,:) = p1 * I1(y_R1,x_R1,:) ...
            + p2 * I2(y_R2,x_R2,:);
```

L'image doit être reconvertie avant sa sauvegarde :

```
imwrite(uint8(Imoscoul), 'mosaique2_coul.jpg');
```

10.4 Hypothèses

- Les deux images ont le même nombre de canaux.
- L'homographie est identique pour les canaux rouge, vert et bleu.
- Aucune conversion d'espace colorimétrique n'est effectuée.
- Aucune interpolation autre que le plus proche voisin n'est utilisée.

---

11 11. Chaîne complète de traitement

Fichier : `TP_mosaique.m`

11.1 Lecture des images

```
Im1 = imread('cancale1.pgm');
Im2 = imread('cancale2.pgm');
Im3 = imread('cancale3.pgm');
```

Versions couleur :

```
Im1_coul = imread('cancale1.jpg');
```

11.2 Détection des points

```
[XY_1,Res_1] = harris(Im1,TailleFenetre,NbPoints,k);
[XY_2,Res_2] = harris(Im2,TailleFenetre,NbPoints,k);
[XY_3,Res_3] = harris(Im3,TailleFenetre,NbPoints,k);
```

11.3 Appariement des images 1 et 2

```
[XY_C1,XY_C2] = apparier_POI( ...
    Im1,XY_1,Im2,XY_2,TailleFenetre,seuil);
```

11.4 Estimation de l'homographie

```
H12 = homographie(XY_C1,XY_C2);
```

Interprétation :

```
H12 transforme les coordonnées de l'image 1
en coordonnées de l'image 2.
```

11.5 Construction des mosaïques à deux images

```
Imos = mosaique(Im1,Im2,H12);
Imosbis = mosaiquebis(Im1,Im2,H12);
Imoscoul = mosaiquecoul(Im1_coul,Im2_coul,H12);
```

11.6 Construction d'une mosaïque à trois images

On commence par estimer la transformation de l'image 2 vers l'image 3 :

```
[XY_C2b,XY_C3] = apparier_POI(Im2,XY_2,Im3,XY_3, ... );
H23 = homographie(XY_C2b,XY_C3);
```

Puis on compose les transformations :

```
image 1 → image 2 → image 3
```

Le code utilise :

```
H23 * H12
```

Une matrice de translation `T12` compense l'origine de la mosaïque :

```
T12 = [1 0 O1x_Rmos;
       0 1 O1y_Rmos;
       0 0 1];
```

Puis :

```
H_123 = (H23 * H12) / T12;
```

La mosaïque finale est :

```
Imos2 = mosaique(Imosbis, Im3, H_123);
Imos2coul = mosaiquecoul(Imoscoul, Im3_coul, H_123);
```

12 12. Concepts MATLAB essentiels

12.1 Dimensions des matrices

Pour `N` points :

```
xy        : N x 2
xy_hom    : N x 3
H         : 3 x 3
H*xy_hom' : 3 x N
```

12.2 Opérateurs terme à terme

Utiliser :

```
.*
./
.^
```

pour les opérations terme à terme.

Exemples :

```
x2 .* x1
A .* B
```

12.3 Opérateurs matriciels

Utiliser :

```
*
/
```

pour la multiplication et la division matricielles.

12.4 Indexation linéaire

```
R(:)
```

transforme une matrice en un vecteur colonne.

12.5 `ind2sub`

```
[Y,X] = ind2sub(size(R), index);
```

renvoie :

```
Y = ligne
X = colonne
```

12.6 Conversion des images

Utiliser :

```
double(I)
```

avant les opérations mathématiques.

Utiliser :

```
uint8(I)
```

avant d'écrire des images standards sur 8 bits.

13 13. Pièges fréquents à l'examen

1. Confondre `(x,y)` avec l'indexation MATLAB `(ligne,colonne)`.
2. Oublier que les pixels sont accessibles avec `I(y,x)`.
3. Utiliser `H` à la place de `inv(H)`.
4. Oublier la normalisation homogène par `z`.
5. Construire incorrectement la matrice `A` de l'estimation de l'homographie.
6. Oublier qu'il faut au minimum quatre correspondances non alignées.
7. Utiliser `var( ... ,2)` au lieu de la variance de population `var( ... ,1,2)`.
8. Oublier d'exclure les points proches des bords.
9. Diviser par zéro pour des voisinages constants.
10. Mélanger les dimensions des images :
    - lignes = hauteur ;
    - colonnes = largeur.
11. Oublier que `reshape` utilise un stockage par colonnes.
12. Affecter des valeurs `uint8` avant une fusion pondérée.
13. Utiliser une homographie estimée à partir de mauvais appariements.
14. Omettre la translation liée à l'origine de la mosaïque.
15. Supposer que l'homographie est symétrique :
    - en général, `H12 ~= inv(H12)`.

---

14 Questionnaire détaillé

14.1 Questionnaire A — Détection de Harris

14.1.1 Questions

1. Que représentent `x` et `y` dans la convention de coordonnées du projet ?
2. Pourquoi l'image d'entrée est-elle convertie avec `double(Im)` ?
3. Que représentent `A`, `B` et `C` ?
4. Écrire l'équation de la réponse de Harris.
5. Pourquoi appliquer un lissage gaussien aux produits des gradients ?
6. Quel est le rôle du paramètre `k` ?
7. Pourquoi la taille de la fenêtre doit-elle normalement être impaire ?
8. Pourquoi attribuer `-Inf` aux réponses des bords ?
9. Expliquer pourquoi `ind2sub` renvoie `[Y,X]`, alors que la sortie est `[X,Y]`.
10. Quel est l'objectif de la suppression des non-maxima locaux ?
11. Que se passe-t-il si `NbPoints` est supérieur au nombre de points valides ?
12. Pourquoi une image uniforme peut-elle ne produire aucun point de Harris utile ?
13. Comparer la réponse de Harris sur un contour et sur un coin.
14. Pourquoi deux pixels voisins peuvent-ils être sélectionnés lorsque la réponse présente un plateau ?
15. Quel effet l'augmentation de `TailleFenetre` a-t-elle sur la détection ?

14.1.2 Exercices pratiques

1. Remplacer la double boucle de suppression des non-maxima locaux par une opération adaptée de traitement d'image.
2. Visualiser séparément les valeurs positives et négatives de `R`.
3. Comparer les points détectés pour :

```
k = 0.04;
k = 0.05;
k = 0.06;
```

1. Expliquer pourquoi les points détectés près d'un bord doivent être supprimés avant l'appariement.

14.2 Questionnaire B — Extraction des voisinages

14.2.1 Questions

1. Quelle est la taille de sortie de `voisinage` pour `N` points et une fenêtre de taille `K` ?
2. Pourquoi l'expression `I(y-K:y+K, x-K:x+K)` est-elle correcte ?
3. Que se passe-t-il si `x-K < 1` ?
4. Pourquoi chaque voisinage est-il stocké sur une ligne ?
5. Dans quel ordre les valeurs de `fenetre(:)` sont-elles parcourues ?
6. Pourquoi les coordonnées des points doivent-elles être entières ?
7. Quelle est la différence entre la largeur de la fenêtre `K` et le demi-rayon `floor(K/2)` ?
8. Pourquoi la fonction suppose-t-elle que les points ne sont pas sur les bords ?
9. Comment la fonction pourrait-elle traiter les points situés près des bords ?
10. Pourquoi est-il utile de convertir `I` en `double` ?

14.2.2 Exercices pratiques

1. Pour `TailleFenetre = 5`, écrire l'intervalle exact de pixels extrait autour de `(x,y)`.
2. Calculer le nombre de valeurs contenues dans un voisinage aplati.
3. Représenter la relation entre :
   - les coordonnées de l'image ;
   - les indices matriciels ;
   - les indices du voisinage aplati.

14.3 Questionnaire C — ZNCC et appariement des points

14.3.1 Questions

1. Que représente `C(i,j)` ?
2. Pourquoi soustraire la moyenne des voisinages ?
3. Pourquoi normaliser la variance avec `var( ... ,1,2)` ?
4. Quel est l'intervalle théorique de la ZNCC ?
5. Que signifie une corrélation égale à `1` ?
6. Que signifie une corrélation égale à `-1` ?
7. Pourquoi le projet applique-t-il `abs(C)` ?
8. Que se passe-t-il lorsqu'un voisinage possède une variance nulle ?
9. Pourquoi la multiplication matricielle permet-elle de calculer efficacement toutes les covariances ?
10. Que représentent `ind1_2` et `ind2_1` ?
11. Expliquer la contrainte de plus proche voisin mutuel.
12. Pourquoi un seuil de `0.9` est-il restrictif ?
13. Quels sont les risques d'un seuil trop faible ?
14. Pourquoi le nombre de lignes de `XY_C1` et `XY_C2` doit-il être identique ?
15. Pourquoi un appariement WTA peut-il être incorrect même si son score est maximal ?

14.3.2 Exercice numérique

Soient :

```
P1 = [1, 2, 3]
P2 = [2, 4, 6]
```

1. Calculer les deux moyennes.
2. Centrer les deux vecteurs.
3. Calculer leur ZNCC.
4. Expliquer pourquoi une modification d'échelle de la luminance ne change pas la corrélation.

14.3.3 Exercice de conception

Concevoir une stratégie d'appariement plus sûre combinant :

- l'appariement mutuel ;
- un seuil de corrélation ;

- un test de rapport ;
- une vérification géométrique.

14.4 Questionnaire D — Estimation de l'homographie

14.4.1 Questions

1. Quelle transformation géométrique une homographie représente-t-elle ?
2. Pourquoi une homographie possède-t-elle huit paramètres indépendants ?
3. Pourquoi quatre correspondances sont-elles théoriquement suffisantes ?
4. Que représente l'équation `A*h = 0` ?
5. Pourquoi chaque correspondance fournit-elle deux lignes ?
6. Expliquer la structure de `bloc_1`.
7. Expliquer la structure de `bloc_2`.
8. Pourquoi sélectionner la dernière colonne de `V` après la SVD ?
9. Pourquoi transposer `reshape(H,3,3)'` ?
10. Qu'est-ce qu'une configuration de points dégénérée ?
11. Pourquoi des points alignés sont-ils insuffisants ?
12. Que se passe-t-il si les correspondances contiennent des valeurs aberrantes ?
13. Pourquoi des coordonnées brutes peuvent-elles provoquer une instabilité numérique ?
14. Qu'est-ce que la normalisation des coordonnées ?
15. Pourquoi l'homographie estimée peut-elle mal transformer les points situés en dehors de la zone observée ?
16. Pourquoi une homographie est-elle définie à un facteur d'échelle près ?
17. Que permet la normalisation de `H(3,3)` ?
18. Comment mesurer la qualité d'une homographie estimée ?

14.4.2 Exercices pratiques

1. Construire les deux lignes de `A` pour :

```
(x1,y1) = (10,20)
(x2,y2) = (15,25)
```

1. Estimer une homographie à partir des quatre coins d'un rectangle.
2. Vérifier que :

```
appliquerHomographie(H, XY_C1)
```

est proche de `XY_C2`. 4. Ajouter une correspondance incorrecte et observer le résultat. 5. Expliquer pourquoi RANSAC améliorerait la robustesse.

14.5 Questionnaire E — Application d'une homographie

14.5.1 Questions

1. Pourquoi convertir les points en coordonnées homogènes ?
2. Pourquoi la multiplication est-elle écrite ainsi :

```
(H * xy1_homogene')'
```

1. Que représente la troisième coordonnée ?
2. Pourquoi chaque ligne doit-elle être divisée par sa propre valeur `z` ?
3. Que se passe-t-il si `z = 0` ?
4. Pourquoi la fonction n'arrondit-elle pas les coordonnées ?
5. Pourquoi la fonction ne vérifie-t-elle pas les limites de l'image ?
6. Quelle est la matrice de l'identité ?
7. Quelle matrice représente une translation de `(tx,ty)` ?
8. Comment vérifier que deux homographies sont inverses l'une de l'autre ?

14.5.2 Exercices pratiques

1. Appliquer la matrice identité à trois points.
2. Appliquer :

```
H = [1 0 10;
     0 1 20;
     0 0 1];
```

1. Expliquer le résultat d'une homographie dont la troisième ligne est :

```
[0.001 0 1]
```

1. Tester le retour aux coordonnées initiales :

```
xy3 = appliquerHomographie(inv(H), ...
       appliquerHomographie(H, xy1));
```

14.6 Questionnaire F — Mosaïques en niveaux de gris

14.6.1 Questions

1. Pourquoi transformer les quatre coins de l'image 2 ?
2. Pourquoi les coordonnées minimale et maximale sont-elles nécessaires ?
3. Pourquoi les limites sont-elles arrondies avec `floor` et `ceil` ?
4. Quel est le rôle de `O1x_Rmos` et `O1y_Rmos` ?
5. Pourquoi construire la mosaïque par déformation inverse ?
6. Qu'est-ce qu'un échantillonnage par plus proche voisin ?
7. Qu'est-ce qui peut créer des trous dans une mosaïque ?
8. Qu'est-ce qui peut créer des images fantômes ?
9. Quelle est la différence entre `mosaiquebis` et `mosaique` ?
10. Que représentent `p1` et `p2` ?
11. Pourquoi le dénominateur de la fusion peut-il devenir problématique ?
12. Pourquoi une fusion fondée uniquement sur la distance horizontale est-elle imparfaite ?
13. Comment l'interpolation bilinéaire améliorerait-elle le résultat ?
14. Pourquoi vérifier les limites de l'image avant d'accéder à `I2(y_R2,x_R2)` ?
15. Pourquoi une mauvaise convention de signe ou de coordonnées peut-elle produire une mosaïque complètement déplacée ?

14.6.2 Exercices de débogage

1. Expliquer ce qui se produit si `H` transforme l'image 1 vers l'image 2, mais que le code la considère comme une transformation de l'image 2 vers l'image 1.
2. Ajouter une vérification confirmant que les quatre coins projetés sont finis.
3. Identifier comment des pixels noirs peuvent apparaître dans le résultat.
4. Proposer une solution fondée sur un masque pour les pixels non affectés.
5. Comparer la copie directe avec la fusion pondérée.

14.7 Questionnaire G — Mosaïques couleur

14.7.1 Questions

1. Pourquoi une image couleur possède-t-elle trois dimensions ?
2. Pourquoi `nbCan` est-il extrait séparément ?
3. Pourquoi convertir les images en `double` avant la fusion ?
4. Pourquoi la sortie est-elle allouée avec :

```
zeros(height,width,channels)
```

1. Pourquoi appliquer la même homographie à chaque canal ?
2. Que se passe-t-il si l'image 1 possède trois canaux et l'image 2 un seul ?
3. Pourquoi retarder la conversion en `uint8` jusqu'à la sauvegarde ?
4. Qu'est-ce qui peut produire des raccords colorés ?
5. Qu'est-ce qui peut provoquer un désalignement entre les canaux ?
6. Comment l'algorithme pourrait-il gérer des images RGBA ?
7. Pourquoi est-il généralement acceptable de fusionner les canaux indépendamment ?
8. Dans quel cas un autre espace colorimétrique pourrait-il améliorer la fusion ?

14.7.2 Exercices pratiques

1. Donner la taille d'une image RGB de dimensions `800 x 1200`.
2. Déterminer les dimensions de :

```
I2(y_R2,x_R2,:)
```

1. Expliquer pourquoi une opération pondérée sur des valeurs `uint8` est dangereuse.
2. Concevoir un masque de validité couleur pour la sortie.

14.8 Questionnaire H — Composition de trois images

14.8.1 Questions

1. Que représente `H12` ?
2. Que représente `H23` ?
3. Quelle transformation permet de passer directement de l'image 1 à l'image 3 ?
4. Pourquoi l'ordre des multiplications matricielles est-il important ?
5. Pourquoi utiliser `H23 * H12` ?
6. Quel est le rôle de `T12` ?
7. Pourquoi l'origine de la mosaïque est-elle différente de l'origine de l'image initiale ?
8. Que peut-il se produire si `T12` est omise ?
9. Pourquoi apparier séparément les images 2 et 3 ?
10. Que se passe-t-il si les deux homographies utilisent des conventions de coordonnées différentes ?
11. Comment étendre la méthode à quatre images ?
12. Pourquoi les erreurs géométriques cumulées peuvent-elles augmenter avec chaque image ajoutée ?
13. Comment un ajustement global des faisceaux pourrait-il améliorer un panorama long ?
14. Pourquoi une zone de recouvrement est-elle nécessaire entre deux images consécutives ?
15. Que se passe-t-il si les images 1 et 3 se recouvrent, mais que l'image 2 ne peut pas être appariée de manière fiable ?

14.8.2 Exercice de composition

Supposons :

```
H12 : image 1 → image 2
H23 : image 2 → image 3
```

Écrire l'expression permettant de transformer un point de l'image 1 vers l'image 3.

Puis expliquer pourquoi :

```
H12 * H23
```

est généralement incorrect.

---

15 Liste de vérification finale

Avant un examen ou une démonstration, vérifier que vous savez expliquer :

- la réponse de Harris et le tenseur de structure ;

- les directions des gradients et l'indexation des images ;
- la suppression des non-maxima locaux ;
- l'extraction des voisinages ;
- la moyenne, la variance, la covariance et la ZNCC ;
- l'appariement par plus proche voisin mutuel ;
- les coordonnées homogènes ;
- l'estimation DLT de l'homographie ;
- la SVD et le vecteur singulier associé à la plus petite valeur singulière ;
- l'inversion d'une homographie ;
- la translation du repère de la mosaïque ;
- la déformation inverse ;
- l'interpolation par plus proche voisin ;
- la fusion dans la zone de recouvrement ;
- la gestion des canaux RGB ;
- la composition des homographies ;
- les configurations dégénérées ;
- le rejet des valeurs aberrantes et RANSAC ;
- les problèmes numériques et les cas limites liés aux bords.